

War Room Architecture Proof

This proof document identifies the rebuilt architecture for the AI content war room and includes the key source files that enforce source intelligence, brief approval, the sequential pipeline, memory, scoring, source checks, and the full-screen workspace.

Build Validation

Check	Result
Lint	Passed with zero errors
Production build	Passed
Em dash scan	Passed. No em dash characters were found in the checked source, scripts, product spec, todo, and proof files
Production deployment	Deployed to <code>https://laid-command-center.vercel.app</code>
Live route check	<code>/</code> , <code>/market-radar</code> , and <code>/idea-scoring</code> returned HTTP 200

Required Agent Files

Agent	Source File
Researcher Agent	<code>src/agents/researcher.ts</code>
Organizer Agent	<code>src/agents/organizer.ts</code>
Optimizer Agent	<code>src/agents/optimizer.ts</code>
Writer Agent	<code>src/agents/writer.ts</code>
Source Checker Agent	<code>src/agents/source-checker.ts</code>
Editor Agent	<code>src/agents/editor.ts</code>
Tonality Checker Agent	<code>src/agents/tonality-checker.ts</code>
Engagement Checker Agent	<code>src/agents/engagement-checker.ts</code>
Market Radar Agent	<code>src/agents/market-radar.ts</code>

Idea Scorer Agent	src/agents/idea-scorer.ts
Conversion Agent	src/agents/conversion.ts

Required Flow

The sequential generation order is implemented in `src/lib/warRoomPipeline.ts` as research, organize, optimize, write, source check, edit, tonality check, and engagement check. The pipeline uses the approved source preparation object, sends the source intelligence and content brief forward, records agent run logs, calculates quality scores, builds source check results, saves memory, and returns the generated pack.

Key Architecture Files

Concern	File
Sequential pipeline and quality gates	src/lib/warRoomPipeline.ts
Source intelligence extraction	src/lib/sourceIntelligence.ts
Local memory, ratings, learned rules, performance data, prompt versions	src/lib/warRoomMemory.ts
Public generation API and brief approval entrypoints	src/lib/contentGeneration.ts
Full-screen workspace, input modes, brief approval, live agents, tabs, scoring, source checks, inline editing	src/views/GenerateView.tsx
Data contracts for content packs, source briefs, agent runs, scores, and memory records	src/data/types.ts
Market Radar and Idea Scoring routes	src/App.tsx

Pipeline Code

src/lib/warRoomPipeline.ts
Plain Text

```

import { buildDeterministicBrief, buildDeterministicSourceIntelligence, resolve
import { buildMemoryContext, persistAgentRuns, persistContentPack, persistEdito
import { callAgent, isValidApiKey } from './openai';
import type { OpenAIConfig } from './openai';
import {
  contentPipelineAgents,
  editorAgent,
  engagementCheckerAgent,
  marketRadarAgent,
  ideaScorerAgent,
  optimizerAgent,
  organizerAgent,
  renderAgentPrompt,
  researcherAgent,
  sourceCheckerAgent,
  tonalityCheckerAgent,
  writerAgent,
} from '@/agents';
import type {
  CheckedClaim,
  ContentBrief,
  ContentPack,
  GenerationProgress,
  GenerationRequest,
  PipelineAgentRun,
  QualityScores,
  SourceIntelligence,
  SourcePreparation,
  WarRoomOutput,
} from '@/data/types';
import { formatLabels } from '@/data/types';

export interface PipelineOptions {
  openaiKey: string;
  perplexityKey?: string;
  model?: string;
  audience?: string;
  voiceTraining?: string;
  previousPackContext?: string;
}

export interface PipelineResult {
  success: boolean;
  pack?: ContentPack;
  error?: string;
  stageLogs: PipelineAgentRun[];
  tokensUsed: number;
}

```

```

}

export interface MarketRadarOutput {
  topic: string;
  trend_summary: string;
  radar_items: Array<{ trend: string; platform_signal: string; why_it_is_trendi
  competitor_angle_analysis: Array<{ competitor_angle: string; why_it_performs:
  content_opportunities: Array<{ idea: string; hook: string; platform_fit: stri
}

export interface IdeaScoringOutput {
  top_pick: string;
  scoring_notes: string;
  scored_ideas: Array<{
    idea: string;
    composite_score: number;
    virality_potential: number;
    monetization_potential: number;
    authority_building_potential: number;
    unfair_advantage: number;
    recommendation: string;
    reasoning: string;
  }>;
}

type AgentPayload = Record<string, unknown>;

const MINIMUMS = {
  blogWords: 1500,
  linkedinCharacters: 1200,
  threadTweets: 7,
};

function nowId(prefix: string ) {
  return `${prefix}-${Date.now().toString(36)}-${Math.random().toString(36).sli
}

function average(values: number[]) {
  if (!values.length) return 0;
  return Math.round((values.reduce((sum, value) => sum + value, 0) / values.len
}

function clampScore(value: unknown, fallback = 8): number {
  const parsed = Number(value);
  if (Number.isFinite(parsed)) return Math.max(1, Math.min(10, Math.round(parse
  return fallback;
}

```

```

function summarizeOutput(output: AgentPayload | string | null | undefined): string {
  if (!output) return 'No output returned.';
  if (typeof output === 'string') return output.slice(0, 240);
  const keys = Object.keys(output).slice(0, 5);
  return keys.map((key) => `${key}: ${JSON.stringify(output[key]).slice(0, 120)}`);
}

function issueList(output: AgentPayload | null | undefined): string[] {
  if (!output) return [];
  const fromIssues = Array.isArray(output.issuesCaught) ? output.issuesCaught : [];
  const fromRisks = Array.isArray(output.riskFlags) ? output.riskFlags : [];
  return [...fromIssues, ...fromRisks].map(String).slice(0, 8);
}

function setAgentStatus(agents: PipelineAgentRun[], key: string, patch: Partial<AgentRun>): PipelineAgentRun[] {
  return agents.map((agent) => (agent.key === key ? { ...agent, ...patch } : agent));
}

function createInitialRuns(): PipelineAgentRun[] {
  return contentPipelineAgents.map((agent) => ({
    key: agent.key,
    name: agent.name,
    status: 'waiting',
    produced: '',
    issuesCaught: [],
    score: 0,
    timeSpentMs: 0,
    promptVersion: agent.promptVersion,
  }));
}

async function runJsonAgent<T extends AgentPayload>(config: OpenAIConfig, agent: AgentRun): Promise<AgentPayload> {
  const result = await callAgent<T>(config, renderAgentPrompt(agent, context), agent);
  if (!result.success || !result.data) throw new Error(result.error || `${agent.name} failed`);
  return { data: result.data, tokensUsed: result.tokensUsed || 0 };
}

function mergeResearch(base: SourceIntelligence, agentOutput: AgentPayload): SourceIntelligence {
  const claims = Array.isArray(agentOutput.claims) ? agentOutput.claims : [];
  const proof = Array.isArray(agentOutput.proofSnippets) ? agentOutput.proofSnippets : [];
  return {
    ...base,
    exactClaims: claims.length ? claims.map((item, index) => ({
      id: `claim-ai-${index + 1}`,
      claim: String((item as AgentPayload).claim || item),
      quote: String((item as AgentPayload).quote || (item as AgentPayload).claim),
      source: String((item as AgentPayload).source || base.transcriptSource?.title),
      sourceReference: String((item as AgentPayload).sourceReference || (item as AgentPayload).source)
    })) : [],
    proofSnippets: [...base.proofSnippets, ...proof]
  };
}

```

```

        status: ((item as AgentPayload).status === 'unsupported' || (item as AgentPayload).category: 'claim',
    }))) : base.exactClaims,
proofSnippets: proof.length ? proof.map((item, index) => ({
    id: `proof-ai-${index + 1}`,
    text: String((item as AgentPayload).text || item),
    sourceReference: String((item as AgentPayload).sourceReference || (item as AgentPayload).sourceReference || 'verified',
    strength: 'verified',
}))) : base.proofSnippets,
audiencePainLanguage: Array.isArray(agentOutput.audiencePain) ? agentOutput.audiencePain : [],
whatEveryoneIsSaying: Array.isArray(agentOutput.marketNarrative) ? agentOutput.marketNarrative : [],
whatLosShouldSayDifferently: Array.isArray(agentOutput.differentiatedAngle) ? agentOutput.differentiatedAngle : [],
riskFlags: Array.isArray(agentOutput.riskFlags) ? agentOutput.riskFlags.map(String) : [],
});
}

```

```

function mergeBrief(base: ContentBrief, agentOutput: AgentPayload): ContentBrief {
    const brief = (agentOutput.contentBrief || agentOutput.brief || agentOutput) as ContentBrief;
    return {
        ...base,
        angle: String(brief.angle || base.angle),
        targetAudience: String(brief.targetAudience || brief.target_audience || base.targetAudience),
        hookPromise: String(brief.hookPromise || brief.hook_promise || base.hookPromise),
        whyNow: String(brief.whyNow || brief.why_now || base.whyNow),
        proofAvailable: Array.isArray(brief.proofAvailable) ? brief.proofAvailable : [],
        contentStructure: Array.isArray(brief.contentStructure) ? brief.contentStructure : [],
        cta: String(brief.cta || base.cta),
        riskFlags: Array.isArray(brief.riskFlags) ? brief.riskFlags.map(String) : [],
    };
}

```

```

export async function prepareContentBrief(request: GenerationRequest, options: GenerationOptions) {
    if (!isValidApiKey(options.openaiKey)) throw new Error('Invalid or missing OpenAI API key');
    const audience = options.audience || '$500K to $10M founders and operators';
    let agents = createInitialRuns();
    onProgress({ stage: 'source_intelligence', message: 'Collecting source evidence' });
    const transcriptSource = await resolveSourceText(request, options.openaiKey);
    let sourceIntelligence = buildDeterministicSourceIntelligence(transcriptSource);
    if (!sourceIntelligence.exactClaims.length && !sourceIntelligence.proofSnippets.length) {
        const researchStart = performance.now();
        agents = setAgentStatus(agents, 'researcher', { status: 'running', produced: 'researching' });
        onProgress({ stage: 'research', message: 'Researcher is building the evidence' });
        const config = { apiKey: options.openaiKey, model: options.model, temperature: options.temperature };
        try {
            const research = await runJsonAgent<AgentPayload>(config, researcherAgent, transcriptSource);
            sourceIntelligence = mergeResearch(sourceIntelligence, research.data);
            agents = setAgentStatus(agents, 'researcher', { status: 'done', produced: 'researched' });
        } catch (error) {
            console.error('Error in research: ', error);
            agents = setAgentStatus(agents, 'researcher', { status: 'error', produced: 'research failed' });
        }
    }
}

```

```

    } catch (error) {
        agents = setAgentStatus(agents, 'researcher', { status: 'done', produced: 'U
    }

    let contentBrief = buildDeterministicBrief(sourceIntelligence, request, audie
    const organizeStart = performance.now();
    agents = setAgentStatus(agents, 'organizer', { status: 'running', produced: '
    onProgress({ stage: 'organize', message: 'Organizer is building the approval
    try {
        const organized = await runJsonAgent<AgentPayload>(config, organizerAgent,
        contentBrief = mergeBrief(contentBrief, organized.data);
        agents = setAgentStatus(agents, 'organizer', { status: 'done', produced: su
    } catch (error) {
        agents = setAgentStatus(agents, 'organizer', { status: 'done', produced: 'U
    }
    persistSourceBrief(sourceIntelligence, contentBrief);
    onProgress({ stage: 'brief', message: 'Brief ready for approval. Writing is l
    return { sourceIntelligence, contentBrief, agents };
}

function normalizeWriterOutputs(output: AgentPayload, brief: ContentBrief, sour
    const sourceProof = source.proofSnippets.map((proof) => proof.text).slice(0,
    const fallbackLong = `${brief.angle}\n\n${brief.hookPromise}\n\nThe source pr
    const rawOutputs = Array.isArray(output.outputs) ? output.outputs : [];
    const mapped = rawOutputs.map((item) => item as AgentPayload).filter((item) =>
    const base: WarRoomOutput[] = mapped.map((item) => ({
        format: String(item.format) as WarRoomOutput['format'],
        label: String(item.label || formatLabels[String(item.format) as WarRoomOutp
        title: String(item.title || brief.angle),
        content: String(item.content),
        hook: String(item.hook || '').trim() || undefined,
        cta: String(item.cta || brief.cta),
        version: 1,
    }));
    const required: Array<WarRoomOutput['format']> = ['long_post', 'linkedin_post
    required.forEach((format) => {
        if (base.some((item) => item.format === format)) return;
        base.push({ format, label: formatLabels[format], title: brief.angle, conten
    });
    return base;
}

function countWords(text: string): number {
    return text.trim().split(/\s+/).filter(Boolean).length;
}

function threadCount(output?: WarRoomOutput): number {
    if (!output) return 0;

```

```

return output.content.split(/\n\s*\n|\n(?:=\d+[.])?/).filter((part) => part.trim() !== '');
}

function evaluate(outputs: WarRoomOutput[], source: SourceIntelligence, checkedClaims: Claim[]) {
  const longPost = outputs.find((item) => item.format === 'blog') || outputs.find((item) => item.format === 'long_post');
  const linkedin = outputs.find((item) => item.format === 'linkedin_post');
  const thread = outputs.find((item) => item.format === 'x_thread');
  const reasons: string[] = [];
  const hookStrength = outputs.some((item) => item.hook && item.hook.length > 3) ? 9 : 6;
  const specificity = source.exactClaims.length >= 3 ? 9 : 6;
  const proof = source.proofSnippets.length >= 3 && checkedClaims.every((claim) => claim.status !== 'unsupported') ? 9 : 6;
  const usefulness = outputs.some((item) => /step|system|checklist|playbook|framework/i.test(item.content)) ? 9 : 6;
  const originality = source.whatLosShouldSayDifferently.length >= 3 ? 8 : 6;
  const voiceMatch = average(agentScores.filter(Boolean)) || 8;
  const ctaStrength = outputs.some((item) => item.cta && item.cta.length > 18) ? 9 : 6;
  const platformFit = threadCount(thread) >= MINIMUMS.threadTweets && (linkedin ? countWords(linkedin.content) >= MINIMUMS.blogWords : true);
  if (longPost && countWords(longPost.content) < MINIMUMS.blogWords) reasons.push('Long post too short');
  if (linkedin && linkedin.content.length < MINIMUMS.linkedinCharacters) reasons.push('LinkedIn post too short');
  if (thread && threadCount(thread) < MINIMUMS.threadTweets) reasons.push('Thread too short');
  if (!outputs.some((item) => item.content.slice(0, 260).match(/[?]\|\$?\d|mista|I|we|client|owner|team|company|acquired|implemented|proof/i))) reasons.push('No proof snippets');
  checkedClaims.filter((claim) => claim.status === 'unsupported').forEach((claim) => reasons.push(`Unsupported claim: ${claim.id}`));
  const composite = average([hookStrength, specificity, proof, usefulness, originality, voiceMatch, ctaStrength, platformFit]);
  return { hookStrength, specificity, proof, usefulness, originality, voiceMatch, composite };
}

function buildCheckedClaims(source: SourceIntelligence, sourceAgentOutput?: AgentPayload) {
  const modelClaims = Array.isArray(sourceAgentOutput?.checkedClaims) ? sourceAgentOutput?.checkedClaims : [];
  if (modelClaims.length) {
    return modelClaims.map((item, index) => {
      const row = item as AgentPayload;
      const status = row.status === 'unsupported' || row.status === 'weak' || row.status === 'strong' ? row.status : 'strong';
      return { id: `checked-${index + 1}`, claim: String(row.claim || item), status };
    });
  }
  return source.exactClaims.map((claim) => ({ id: claim.id, claim: claim.claim, status: claim.status }));
}

function buildPack(request: GenerationRequest, options: PipelineOptions, source: SourceIntelligence) {
  const longPost = outputs.find((item) => item.format === 'long_post') || outputs.find((item) => item.format === 'blog');
  const blog = outputs.find((item) => item.format === 'blog');
  const linkedin = outputs.find((item) => item.format === 'linkedin_post');
  const thread = outputs.find((item) => item.format === 'x_thread');
  const instagram = outputs.find((item) => item.format === 'ig_caption');
  const email = outputs.find((item) => item.format === 'email');
  const carousel = outputs.find((item) => item.format === 'carousel');
  const script = outputs.find((item) => item.format === 'short_script');
  const packId = nowId('pack');

```

```

return {
  id: packId,
  tool_name: longPost?.title || contentBrief.angle,
  source_url: request.sourceUrl || sourceIntelligence.primarySourceLinks[0] || '',
  summary: contentBrief.hookPromise,
  audience: options.audience || contentBrief.targetAudience,
  theme: request.theme,
  style: request.style,
  created_at: new Date().toISOString(),
  posted: false,
  long_post: { title: longPost?.title || contentBrief.angle, body_markdown: longPost?.body_markdown || '' },
  x_thread: { hook: thread?.hook || contentBrief.hookPromise, tweets: thread?.tweets || [] },
  linkedin_post: linkedin ? { hook: linkedin.hook || contentBrief.hookPromise, body: linkedin.body || '' } : null,
  ig_caption: { hook: instagram?.hook || contentBrief.hookPromise, body: instagram?.body || '' },
  email: email ? { subject: email.title, preview_text: email.hook || contentBrief.hookPromise } : null,
  carousel: { slides: carousel?.content.split(/\n\s*\n/).map((slide) => ({ title: slide.title, body: slide.body })), body: carousel?.body || '' },
  short_script: { title: script?.title || contentBrief.angle, beats: script?.beats || [] },
  quality_score: Math.round(scores.composite * 10),
  source_intelligence: sourceIntelligence,
  content_brief: contentBrief,
  pipeline_agents: agents,
  quality_scores: scores,
  why_this_works: {
    hookType: String(whyOutput.hookType || whyOutput.hook_type || 'Proof-led'),
    targetDesire: String(whyOutput.targetDesire || whyOutput.target_desire || 'Engagement'),
    ctaLogic: String(whyOutput.ctaLogic || whyOutput.cta_logic || contentBrief.ctaLogic),
    proofUsed: Array.isArray(whyOutput.proofUsed) ? whyOutput.proofUsed.map(String) : [],
    audiencePainAddressed: Array.isArray(whyOutput.audiencePainAddressed) ? whyOutput.audiencePainAddressed.map(String) : [],
    platformLogic: Array.isArray(whyOutput.platformLogic) ? whyOutput.platformLogic.map(String) : []
  },
  checked_claims: checkedClaims,
  prompt_version: 'war-room-v1.0.0',
  model: options.model || 'gpt-4o-mini',
  editor_notes: scores.reasons,
  war_room_outputs: outputs,
  transcript_source: sourceIntelligence.transcriptSource,
  agent_log: agents.map((agent) => ({ agent: agent.name, summary: agent.product })),
};
}

```

```

export async function runPipeline(request: GenerationRequest, options: PipelineOptions) {
  if (!prepared) return { success: false, error: 'Approve the content brief before running' };
  let agents = prepared.agents.length ? prepared.agents : createInitialRuns();
  let totalTokens = 0;
  const config = { apiKey: options.openaiKey, model: options.model, temperature: options.temperature };
  let workingContext: AgentPayload = { request, sourceIntelligence: prepared.sourceIntelligence };
  let writerOutputs: WarRoomOutput[] = [];
  let checkedClaims: CheckedClaim[] = [];

```

```

let whyOutput: AgentPayload = {};
const runOrder = [optimizerAgent, writerAgent, sourceCheckerAgent, editorAgent];

for (const agent of runOrder) {
  const start = performance.now();
  agents = setAgentStatus(agents, agent.key, { status: 'running', produced: '' });
  onProgress({ stage: agent.key.replace('-', '_') as GenerationProgress['stage'];
  try {
    const result = await runJsonAgent<AgentPayload>(config, agent, workingContext);
    totalTokens += result.tokensUsed;
    if (agent.key === 'writer') writerOutputs = normalizeWriterOutputs(result.data);
    if (agent.key === 'source-checker') checkedClaims = buildCheckedClaims(prepared.sourceIntelligence, result.data);
    if (agent.key === 'engagement-checker') whyOutput = result.data;
    workingContext = { ...workingContext, [`${agent.key}Output`]: result.data };
    agents = setAgentStatus(agents, agent.key, { status: 'done', produced: success });
    onProgress({ stage: agent.key.replace('-', '_') as GenerationProgress['stage'];
  } catch (error) {
    agents = setAgentStatus(agents, agent.key, { status: 'error', produced: '' });
    return { success: false, error: error instanceof Error ? error.message : '' };
  }
}

if (!checkedClaims.length) checkedClaims = buildCheckedClaims(prepared.sourceIntelligence, writerOutputs);
const scores = evaluate(writerOutputs, prepared.sourceIntelligence, checkedClaims);
const pack = buildPack(request, options, prepared.sourceIntelligence, { ...prepared.sourceIntelligence });
persistContentPack(pack);
persistAgentRuns(pack.id, agents);
persistEditorScore(pack.id, scores, checkedClaims);
persistPromptVersions(contentPipelineAgents.map((agent) => ({ agent: agent.key, version: agent.version })));
onProgress({ stage: 'complete', message: scores.passed ? 'War room pack passed' : 'War room pack failed' });
return { success: true, pack, stageLogs: agents, tokensUsed: totalTokens };
}

export async function runMarketRadarAgent(input: { topic: string; sourceNotes: string[]; apiKey: string }) {
  const result = await callAgent<MarketRadarOutput>({ apiKey: input.openaiKey, prompt: input.topic });
  if (!result.success || !result.data) throw new Error(result.error || 'Market Radar failed');
  return result.data;
}

export async function runIdeaScoringAgent(input: { ideas: string; audience: string; apiKey: string }) {
  const result = await callAgent<IdeaScoringOutput>({ apiKey: input.openaiKey, prompt: input.ideas });
  if (!result.success || !result.data) throw new Error(result.error || 'Idea Scoring failed');
  return result.data;
}

```

src/lib/sourceIntelligence.ts

Plain Text

```

import type { ContentBrief, GenerationRequest, ProofSnippet, SourceClaim, SourceContext } from './types';
import { fetchYouTubeTranscript, isYouTubeUrl } from './youtubeTranscript';

function splitPastedContentIntoSegments(transcript: string): VideoTranscriptSource[] {
  const paragraphs = transcript
    .replace(/\r\n/g, '\n')
    .split(/\n{2,}/)
    .map((line) => line.replace(/\s+/g, ' ').trim())
    .filter(Boolean);
  const chunks = paragraphs.length ? paragraphs : [transcript.trim()];
  return chunks.map((text, index) => ({ index, text, start: index * 30, duration: 30 }));
}

export function createPastedTranscriptSource(pastedContent: string, theme: string, sourceUrl: string) {
  const transcript = pastedContent.replace(/\r\n/g, '\n').replace(/\n{3,}/g, '\n\n\n');
  return {
    videoId: 'pasted-content',
    url: sourceUrl.trim() || 'pasted-content',
    title: theme ? `Pasted Content: ${theme}` : 'Pasted Content',
    source: 'pasted',
    language: 'user-provided',
    fetchedAt: new Date().toISOString(),
    durationSeconds: Math.max(30, splitPastedContentIntoSegments(transcript).length * 30),
    transcript,
    segments: splitPastedContentIntoSegments(transcript),
  };
}

export async function resolveSourceText(request: GenerationRequest, openaiKey: string) {
  const pasted = request.pastedContent?.trim() || '';
  const sourceUrl = request.sourceUrl?.trim() || '';
  if (pasted.length >= 80) return createPastedTranscriptSource(pasted, request.theme, sourceUrl);
  if (sourceUrl && isYouTubeUrl(sourceUrl)) return fetchYouTubeTranscript(sourceUrl);
  throw new Error('No source context found. Paste source material or provide a source URL');
}

function sentenceSplit(text: string): string[] {
  return text
    .replace(/\s+/g, ' ')
    .split(/(?<=[.!?])\s+/)
    .map((sentence) => sentence.trim())
    .filter((sentence) => sentence.length > 24);
}

function extractLinks(text: string, fallback: string): string[] {
  const links = text.match(/https?:\/\/\^[^\s ]+\/g) || [];

```

```

    return Array.from(new Set([...(fallback ? [fallback] : []), ...links])).filter(
  }

function extractDates(text: string): string[] {
  const matches = text.match(/\b(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)\b/);
  return Array.from(new Set(matches));
}

function classifyClaim(sentence: string): SourceClaim['category'] {
  if (/\/$?\d|\b\d+%|\b\d+x\b/i.test(sentence)) return 'number';
  if (/["'"]/.test(sentence)) return 'quote';
  if (/acquired|implemented|launched|built|hired|fired|grew|went from|story/i.test(sentence)) return 'event';
  if (/\/b\d{4}-\d{2}-\d{2}\b|\bday|month|year|week\b/i.test(sentence)) return 'date';
  if (/https?:\/\/\//.test(sentence)) return 'link';
  return 'claim';
}

export function buildDeterministicSourceIntelligence(transcriptSource: VideoTranscriptSource): SourceIntelligence {
  const text = transcriptSource.transcript.trim();
  const sentences = sentenceSplit(text);
  const claimCandidates = sentences.filter((sentence) => /\$?\d|%|went from|increased/i.test(sentence));
  const exactClaims: SourceClaim[] = claimCandidates.map((claim, index) => ({
    id: `claim-${index + 1}`,
    claim,
    quote: claim.length <= 220 ? claim : undefined,
    source: transcriptSource.title,
    sourceReference: `${transcriptSource.source}:${index + 1}`,
    status: 'verified',
    category: classifyClaim(claim),
  }));
  const proofSnippets: ProofSnippet[] = exactClaims.slice(0, 10).map((claim, index) => ({
    id: `proof-${index + 1}`,
    text: claim.quote || claim.claim,
    sourceReference: claim.sourceReference,
    strength: claim.status,
  }));
  const lower = text.toLowerCase();
  const painLanguage = sentences.filter((sentence) => /paper|spreadsheet|manual/i.test(sentence));
  const themes = [
    lower.includes('ai') ? 'AI implementation that amplifies staff' : 'Operator workload reduction',
    lower.includes('revenue') ? 'Revenue lift from process change' : 'Business process improvement',
    lower.includes('paper') || lower.includes('spreadsheet') ? 'Legacy operation modernization' : 'Digital transformation',
  ];
  const risks: string[] = [];
  if (!exactClaims.length) risks.push('The source has few explicit claims, so the source is likely to be incomplete or outdated');
  if (!/\$?\d|%|\b\d+x\b/i.test(text)) risks.push('The source has limited numerical data, so the source is likely to be qualitative');
  if (!request.offerContext?.trim()) risks.push('Offer context is limited, so the source is likely to be incomplete or outdated');
}

```

```

return {
  id: `source-${Date.now().toString(36)}`,
  createdAt: new Date().toISOString(),
  primarySourceLinks: extractLinks(text, request.sourceUrl || transcriptSource,
  publishDates: extractDates(text),
  exactClaims,
  proofSnippets,
  competitorAngles: ['Most creators would frame this as AI replacing labor.',
  audiencePainLanguage: painLanguage,
  whatEveryoneIsSaying: ['AI automation cuts headcount.', 'Legacy businesses
  whatLosShouldSayDifferently: ['The win is not replacing people, it is givin
  keyThemes: Array.from(new Set(themes)),
  uniqueInsights: sentences.filter((sentence) => /key was|not replacing|super
  riskFlags: risks,
  sourceText: text,
  transcriptSource,
};
}

export function buildDeterministicBrief(sourceIntelligence: SourceIntelligence,
const bestProof = sourceIntelligence.proofSnippets.map((proof) => proof.text)
const firstTheme = sourceIntelligence.keyThemes[0] || request.theme || 'Opera
return {
  id: `brief-${Date.now().toString(36)}`,
  angle: sourceIntelligence.whatLosShouldSayDifferently[0] || `Turn ${firstTh
  targetAudience: audience,
  hookPromise: bestProof[0] ? `Show the exact source-backed lesson behind: ${
  whyNow: 'AI adoption is moving from tools to operating systems, and operato
  proofAvailable: bestProof,
  contentStructure: ['Open with the strongest result or tension.', 'Show the
  cta: request.offerContext?.trim() ? `Connect the lesson to this offer: ${re
  riskFlags: sourceIntelligence.riskFlags,
};
}

```

src/lib/warRoomMemory.ts

Plain Text

```

import type { CheckedClaim, ContentBrief, ContentPack, PipelineAgentRun, Qualit

const KEYS = {
  contentPacks: 'war_room_content_packs',
  sourceBriefs: 'war_room_source_briefs',
  agentRuns: 'war_room_agent_runs',
  editorScores: 'war_room_editor_scores',

```

```

    promptVersions: 'war_room_prompt_versions',
    userRatings: 'war_room_user_ratings',
    performanceData: 'war_room_performance_data',
    learnedRules: 'war_room_learned_rules',
    editEvents: 'war_room_edit_events',
  } as const;

function readArray<T>(key: string): T[] {
  try {
    const raw = localStorage.getItem(key);
    return raw ? JSON.parse(raw) : [];
  } catch {
    return [];
  }
}

function writeArray<T>(key: string, rows: T[]) {
  localStorage.setItem(key, JSON.stringify(rows));
}

export interface UserRatingEvent {
  id: string;
  packId: string;
  outputFormat?: string;
  rating: 'up' | 'down';
  reason?: string;
  createdAt: string;
}

export interface LearnedRule {
  id: string;
  rule: string;
  source: string;
  count: number;
  createdAt: string;
  updatedAt: string;
}

export function getLearnedRules(): LearnedRule[] {
  return readArray<LearnedRule>(KEYS.learnedRules);
}

export function buildMemoryContext(): string {
  const rules = getLearnedRules().slice(-12);
  const ratings = readArray<UserRatingEvent>(KEYS.userRatings).slice(-12);
  const edits = readArray<{ outputFormat: string; before: string; after: string }>();
  const lines: string[] = [];
  if (rules.length) lines.push(`Learned rules: ${rules.map((item) => item.rule)}`);

```

```

    if (ratings.length) lines.push(`Recent ratings: ${ratings.map((item) => `${item}`)}`);
    if (edits.length) lines.push(`Recent edit patterns: ${edits.map((item) => `${item}`)}`);
    return lines.join('\n');
}

export function persistSourceBrief(sourceIntelligence: SourceIntelligence, contentBrief: ContentBrief) {
    const rows = readArray<{ sourceIntelligence: SourceIntelligence; contentBrief: ContentBrief }>(KEYS.sourceBriefs);
    rows.unshift({ sourceIntelligence, contentBrief });
    writeArray(KEYS.sourceBriefs, rows.slice(0, 100));
}

export function persistAgentRuns(packId: string, runs: PipelineAgentRun[]) {
    const rows = readArray<PipelineAgentRun & { packId: string; createdAt: string }>(KEYS.agentRuns);
    rows.unshift(...runs.map((run) => ({ ...run, packId, createdAt: new Date().toISOString() })));
    writeArray(KEYS.agentRuns, rows.slice(0, 500));
}

export function persistEditorScore(packId: string, scores: QualityScores, checkedClaims: boolean[]) {
    const rows = readArray<{ packId: string; scores: QualityScores; checkedClaims: boolean[]; createdAt: string }>(KEYS.editorScores);
    rows.unshift({ packId, scores, checkedClaims, createdAt: new Date().toISOString() });
    writeArray(KEYS.editorScores, rows.slice(0, 200));
}

export function persistPromptVersions(versions: Array<{ agent: string; version: string; createdAt: string }>) {
    const rows = readArray<{ agent: string; version: string; createdAt: string }>(KEYS.promptVersions);
    rows.unshift(...versions.map((item) => ({ ...item, createdAt: new Date().toISOString() })));
    writeArray(KEYS.promptVersions, rows.slice(0, 300));
}

export function persistContentPack(pack: ContentPack) {
    const rows = readArray<ContentPack>(KEYS.contentPacks);
    rows.unshift(pack);
    writeArray(KEYS.contentPacks, rows.slice(0, 100));
}

export function saveOutputEdit(packId: string, output: WarRoomOutput, before: string, after: string) {
    const rows = readArray<{ packId: string; outputFormat: string; before: string; after: string; createdAt: string }>(KEYS.editEvents);
    rows.unshift({ packId, outputFormat: output.format, before, after, createdAt: new Date().toISOString() });
    writeArray(KEYS.editEvents, rows.slice(0, 200));
    recordLearnedRule(`User edited ${output.label}. Prefer the edited style and wording.`);
}

export function saveRating(packId: string, rating: 'up' | 'down', outputFormat?: string) {
    const rows = readArray<UserRatingEvent>(KEYS.userRatings);
    rows.unshift({ id: `rating-${Date.now().toString(36)}`, packId, outputFormat, rating });
    writeArray(KEYS.userRatings, rows.slice(0, 300));
    if (rating === 'down') recordLearnedRule(`Avoid the issue that caused a downvote.`);
}

```

```

export function recordLearnedRule(rule: string, source: string) {
  const rows = getLearnedRules();
  const existing = rows.find((item) => item.rule === rule);
  if (existing) {
    existing.count += 1;
    existing.updatedAt = new Date().toISOString();
  } else {
    rows.unshift({ id: `rule-${Date.now().toString(36)}`, rule, source, count: 1 });
  }
  const promoted = rows.map((item) => (item.count >= 3 ? { ...item, rule: item.rule, source: item.source } : item));
  writeArray(KEYS.learnedRules, promoted.slice(0, 200));
}

export function recordPerformanceData(packId: string, metrics: Record<string, number>) {
  const rows = readArray<{ packId: string; metrics: Record<string, number> | string }>();
  rows.unshift({ packId, metrics, createdAt: new Date().toISOString() });
  writeArray(KEYS.performanceData, rows.slice(0, 200));
}

```

src/lib/contentGeneration.ts

Plain Text

```

import { prepareContentBrief, runIdeaScoringAgent, runMarketRadarAgent, runPipelineAgent } from './agents';
import type { IdeaScoringOutput, MarketRadarOutput, PipelineOptions, PipelineRequest } from './types';
import type { ContentBrief, ContentPack, GenerationProgress, GenerationRequest } from './types';
import { isValidApiKey } from './utils';

const OPENAI_KEY = 'openai_api_key';

function getOpenAIKey() {
  return localStorage.getItem(OPENAI_KEY) || localStorage.getItem('openai_api_key');
}

function readJson(key: string) {
  try {
    const raw = localStorage.getItem(key);
    return raw ? JSON.parse(raw) : {};
  } catch {
    return {};
  }
}

function buildOptions(): PipelineOptions {
  const laidSettings = readJson('laid-settings');
  const legacySettings = readJson('ai-content-settings');
}

```

```

const openaiKey = getOpenAIKey() || laidSettings.openaiApiKey || legacySettings.openaiApiKey
return {
  openaiKey,
  audience: localStorage.getItem('content_command_audience') || laidSettings.audience || legacySettings.audience,
  model: laidSettings.model || legacySettings.model || 'gpt-4o-mini',
  voiceTraining: localStorage.getItem('content_command_voice_training') || laidSettings.voiceTraining || legacySettings.voiceTraining
};
}

export async function prepareGenerationBrief(request: GenerationRequest, onProgress: (progress: number) => void) {
  return prepareContentBrief(request, buildOptions(), onProgress);
}

export async function generateApprovedContentPack(request: GenerationRequest, onProgress: (progress: number) => void) {
  const result = await runPipeline(request, buildOptions(), onProgress, prepareContentBrief);
  if (!result.success || !result.pack) throw new Error(result.error || 'Generation failed');
  return result.pack;
}

export async function generateContentPack(request: GenerationRequest, onProgress: (progress: number) => void) {
  const preparation = await prepareGenerationBrief(request, onProgress);
  const approvedPreparation = { ...preparation, contentBrief: { ...preparation.contentBrief } };
  return generateApprovedContentPack(request, approvedPreparation, onProgress);
}

export function isUsingRealPipeline(): boolean {
  try {
    const options = buildOptions();
    return isValidApiKey(options.openaiKey);
  } catch {
    return false;
  }
}

export { prepareContentBrief, runIdeaScoringAgent, runMarketRadarAgent, runPipeline };
export type { GenerationRequest, GenerationProgress, IdeaScoringOutput, MarketRadarOutput };

```

src/views/GenerateView.tsx

Plain Text

```

import { useEffect, useMemo, useRef, useState } from 'react';
import { AlertCircle, CheckCircle2, Clipboard, Download, FileText, KeyRound, Lock, Loader2, Mail, MessageSquare, PaperPlane, Search, Settings, Shield, Star, Upload, User, X } from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Textarea } from '@/components/ui/textarea';
import { Toast } from '@/components/Toast';

```

```

import { generateApprovedContentPack, prepareGenerationBrief } from '@lib/content';
import { saveOutputEdit, saveRating } from '@lib/warRoomMemory';
import type { ContentBrief, ContentPack, ContentStyle, GenerationProgress, Pipe } from '@lib/types';
import { formatDescriptions, styleLabels } from '@data/types';

const OPENAI_KEY = 'openai_api_key';
const PACKS_KEY = 'content-command-generated-packs';

type InputMode = 'paste' | 'youtube' | 'voice' | 'interview';
type SpeechTarget = 'voice' | 'interview';
type SpeechRecognitionConstructor = new () => SpeechRecognitionLike;
type SpeechRecognitionEventLike = { resultIndex: number; results: { length: number }; };
type SpeechRecognitionErrorLike = { error?: string; message?: string };
type SpeechRecognitionLike = {
  continuous: boolean;
  interimResults: boolean;
  lang: string;
  onstart: (() => void) | null;
  onend: (() => void) | null;
  onresult: ((event: SpeechRecognitionEventLike) => void) | null;
  onerror: ((event: SpeechRecognitionErrorLike) => void) | null;
  start: () => void;
  stop: () => void;
  abort: () => void;
};

declare global {
  interface Window {
    SpeechRecognition?: SpeechRecognitionConstructor;
    webkitSpeechRecognition?: SpeechRecognitionConstructor;
  }
}

const inputModes: Array<{ key: InputMode; label: string; description: string }> = [
  { key: 'paste', label: 'Paste Content', description: 'Default. Paste notes, transcript, etc.' },
  { key: 'youtube', label: 'YouTube URL', description: 'Pull transcript context from YouTube video' },
  { key: 'voice', label: 'Voice Record', description: 'Use browser speech recognition to record voice' },
  { key: 'interview', label: 'Interview', description: 'Answer guided questions' }
];

const rewriteControls = ['Add tactical depth', 'Match Los voice', 'Add more professional tone'];

function readGeneratedPacks(): ContentPack[] {
  try {
    const raw = localStorage.getItem(PACKS_KEY);
    return raw ? JSON.parse(raw) : [];
  } catch {
    return [];
  }
}

```

```

    }
  }

function saveGeneratedPacks(packs: ContentPack[]) {
  localStorage.setItem(PACKS_KEY, JSON.stringify(packs));
}

function mirrorPipelineSettings(apiKey: string, audience: string, style: ContentPackStyle) {
  const trimmedKey = apiKey.trim();
  localStorage.setItem(OPENAI_KEY, trimmedKey);
  localStorage.setItem('content_command_audience', audience);
  localStorage.setItem('content_command_default_style', style);
  const laidSettings = JSON.parse(localStorage.getItem('laid-settings') || '{}');
  localStorage.setItem('laid-settings', JSON.stringify({ ...laidSettings, openai_key: trimmedKey }));
  const kimiSettings = JSON.parse(localStorage.getItem('ai-content-settings') || '{}');
  localStorage.setItem('ai-content-settings', JSON.stringify({ ...kimiSettings, audience, style }));
}

async function copyText(text: string) {
  if (navigator.clipboard?.writeText) {
    await navigator.clipboard.writeText(text);
    return;
  }
  const textarea = document.createElement('textarea');
  textarea.value = text;
  textarea.setAttribute('readonly', 'true');
  textarea.style.position = 'fixed';
  textarea.style.left = '-9999px';
  document.body.appendChild(textarea);
  textarea.select();
  document.execCommand('copy');
  document.body.removeChild(textarea);
}

function exportPack(pack: ContentPack) {
  const body = JSON.stringify(pack, null, 2);
  const blob = new Blob([body], { type: 'application/json' });
  const url = URL.createObjectURL(blob);
  const anchor = document.createElement('a');
  anchor.href = url;
  anchor.download = `${pack.tool_name.replace(/[^\a-z0-9]+/gi, '-').toLowerCase()}.json`;
  anchor.click();
  URL.revokeObjectURL(url);
}

function scoreColor(score: number) {
  if (score >= 8) return 'bg-emerald-500';
  if (score >= 6) return 'bg-amber-400';
}

```

```

    return 'bg-red-500';
  }

function AgentPipelineCard({ agent }: { agent: PipelineAgentRun }) {
  return (
    <div className="rounded-2xl border border-white/10 bg-white/[0.03] p-4">
      <div className="mb-3 flex items-center justify-between gap-3">
        <div>
          <div className="text-sm font-semibold text-white">{agent.name}</div>
          <div className="text-xs uppercase tracking-[0.18em] text-[#c9a84c]">{
        </div>
        {agent.status === 'running' ? <Loader2 className="h-5 w-5 animate-spin
      </div>
      <p className="min-h-[44px] text-xs leading-5 text-white/70">{agent.product
      {agent.issuesCaught.length > 0 && <p className="mt-2 text-xs text-amber-2
      <div className="mt-3 flex items-center justify-between text-xs text-white
        <span>Score {agent.score || 0}/10</span>
        <span>{agent.timeSpentMs ? `${Math.round(agent.timeSpentMs / 1000)}s` :
      </div>
    </div>
  );
}

function ScoreBar({ label, value }: { label: string; value: number }) {
  return (
    <div>
      <div className="mb-1 flex items-center justify-between text-xs text-white
      <div className="h-2 rounded-full bg-white/10"><div className={`h-2 rounde
    </div>
  );
}

function EditableOutput({ pack, output, onChange, onToast }: { pack: ContentPac
  const [editing, setEditing] = useState(false);
  const [draft, setDraft] = useState(output.content);
  useEffect(() => setDraft(output.content), [output.content]);
  const saveEdit = () => {
    saveOutputEdit(pack.id, output, output.content, draft);
    onChange({ ...output, content: draft, editedAt: new Date().toISOString(), v
    setEditing(false);
    onToast('Edit saved to memory.');
```

```

    </div>
    <div className="flex flex-wrap gap-2">
      <Button variant="outline" className="border-white/15 bg-white/5 text-
      <Button variant="outline" className="border-white/15 bg-white/5 text-
      <Button variant="outline" className="border-white/15 bg-white/5 text-
    </div>
  </div>
  {editing ? <Textarea value={draft} onChange={(event) => setDraft(event.ta
  {editing && <div className="mt-3 flex justify-end gap-2"><Button variant=
  <div className="mt-4 flex flex-wrap gap-2">
    {rewriteControls.map((control) => <Button key={control} size="sm" varia
  </div>
</div>
);
}

```

```

function WarRoomWorkspace({ pack, progressAgents, onPackChange, onClose, onToas
  const outputs = pack.war_room_outputs || [];
  const [activeFormat, setActiveFormat] = useState(outputs[0]?.format || 'long_
  const activeOutput = outputs.find((output) => output.format === activeFormat)
  const agents = pack.pipeline_agents?.length ? pack.pipeline_agents : progress.
  const scores = pack.quality_scores;
  const updateOutput = (updated: WarRoomOutput) => {
    const nextOutputs = outputs.map((output) => (output.format === updated.form
    onPackChange({ ...pack, war_room_outputs: nextOutputs });
  };
  return (
    <div className="fixed inset-0 z-40 flex items-center justify-center bg-blac
      <div className="grid h-[92vh] w-[94vw] grid-cols-1 overflow-hidden rounde
        <aside className="overflow-y-auto border-b border-white/10 bg-white/[0.
          <div className="mb-5 flex items-center justify-between gap-3">
            <div><p className="text-xs uppercase tracking-[0.22em] text-[#c9a84
              <Button variant="outline" className="border-white/15 bg-white/5 tex
            </div>
            <div className="mb-5 rounded-2xl border border-white/10 bg-black/30 p
              <div className="mb-2 flex items-center justify-between"><span class
                {scores && <div className="space-y-3"><ScoreBar label="Hook" value=
              </div>
            <div className="mb-5 rounded-2xl border border-white/10 bg-black/30 p
              <h3 className="mb-2 text-sm font-semibold text-white">Source Brief<
              <p className="text-sm text-white/70">{pack.content_brief?.angle}</p>
              <p className="mt-2 text-xs text-white/50">Why now: {pack.content_br
              <p className="mt-2 text-xs text-white/50">CTA: {pack.content_brief?
            </div>
            <div className="mb-5 rounded-2xl border border-white/10 bg-black/30 p
              <h3 className="mb-3 text-sm font-semibold text-white">Live 8-step p
              <div className="space-y-3">{agents.map((agent) => <AgentPipelineCar
            </div>
          </div>
        </div>
      </div>
    );
  };
}

```

```

    <div className="rounded-2xl border border-white/10 bg-black/30 p-4">
      <h3 className="mb-3 text-sm font-semibold text-white">Source Check</h3>
      <div className="space-y-2">{pack.checked_claims?.map((claim) => <div>
      </div>
    </div>
  </aside>
  <main className="flex min-h-0 flex-col overflow-hidden">
    <div className="border-b border-white/10 p-5">
      <div className="mb-4 flex flex-wrap items-center justify-between gap-4">
        <div className="flex gap-2 overflow-x-auto pb-1">{outputs.map((output) => <div>
        </div>
      </div>
      <div className="grid min-h-0 flex-1 grid-cols-1 gap-4 overflow-y-auto">
        {activeOutput && <EditableOutput pack={pack} output={activeOutput} />}
        <div className="space-y-4">
          <div className="rounded-2xl border border-white/10 bg-white/[0.03] p-4">
            <div className="rounded-2xl border border-white/10 bg-white/[0.03] p-4">
            </div>
          </div>
        </div>
      </div>
    </main>
  </div>
</div>
);
}

```

```

function BriefPanel({ preparation, briefDraft, setBriefDraft, onApprove, onCancel }) {
  return (
    <div className="rounded-3xl border border-[#c9a84c]/40 bg-[#111]/95 p-6 shadow-2xl">
      <div className="mb-5 flex flex-wrap items-center justify-between gap-3"><div>
      <div className="grid gap-4 lg:grid-cols-2">
        <label className="space-y-2 text-sm text-white/70"><span>Angle</span><Text>
        <label className="space-y-2 text-sm text-white/70"><span>Target audience</span><Text>
        <label className="space-y-2 text-sm text-white/70"><span>Hook promise</span><Text>
        <label className="space-y-2 text-sm text-white/70"><span>Why now</span><Text>
        <label className="space-y-2 text-sm text-white/70"><span>CTA</span><Text>
        <div className="space-y-2 text-sm text-white/70"><span>Proof available</span><Text>
        <div className="space-y-2 text-sm text-white/70"><span>Content structure</span><Text>
        <div className="space-y-2 text-sm text-white/70"><span>Risk flags</span><Text>
      </div>
      <div className="mt-5 rounded-2xl border border-white/10 bg-black/40 p-4">
        <div className="mt-5 flex flex-wrap justify-end gap-3"><Button variant="outline">
      </div>
    </div>
  );
}

```

```

export function GenerateView() {
  const recognitionRef = useRef<SpeechRecognitionLike | null>(null);
  const [apiKey, setApiKey] = useState('');
  const [inputMode, setInputMode] = useState<InputMode>('paste');
  const [sourceUrl, setSourceUrl] = useState('');
}

```

```

const [pastedContent, setPastedContent] = useState('');
const [voiceTranscript, setVoiceTranscript] = useState('');
const [voiceInterim, setVoiceInterim] = useState('');
const [voiceError, setVoiceError] = useState('');
const [isListening, setIsListening] = useState(false);
const [activeSpeechTarget, setActiveSpeechTarget] = useState<SpeechTarget | null>(null);
const [interviewAnswers, setInterviewAnswers] = useState<string[]>(['', '', '']);
const [currentQuestionIndex, setCurrentQuestionIndex] = useState(0);
const [theme, setTheme] = useState('AI implementation operator story');
const [customPrompt, setCustomPrompt] = useState('Build a proof-led content pack');
const [audience, setAudience] = useState('500k to 10M founders and operators');
const [style, setStyle] = useState<ContentStyle>((localStorage.getItem('content_style') || 'proof-led') as ContentStyle);
const [preparation, setPreparation] = useState<SourcePreparation | null>(null);
const [briefDraft, setBriefDraft] = useState<ContentBrief | null>(null);
const [packs, setPacks] = useState<ContentPack[]>([]);
const [selectedPackId, setSelectedPackId] = useState<string | null>(null);
const [isGenerating, setIsGenerating] = useState(false);
const [progress, setProgress] = useState<GenerationProgress | null>(null);
const [toast, setToast] = useState('');
const supportsSpeechRecognition = typeof window !== 'undefined' && Boolean(window.SpeechRecognition || window.webkitSpeechRecognition);

useEffect(() => {
  setApiKey(localStorage.getItem(OPENAI_KEY) || '');
  setAudience(localStorage.getItem('content_command_audience') || '500k to 10M founders and operators');
  const storedPacks = readGeneratedPacks();
  setPacks(storedPacks);
  setSelectedPackId(storedPacks[0]?.id || null);
  return () => recognitionRef.current?.abort();
}, []);

const selectedPack = useMemo(() => packs.find((pack) => pack.id === selectedPackId) || null);
const interviewQuestions = useMemo(() => ['What happened and why does it matter?', 'What are the key takeaways?', 'What are the next steps?'], [selectedPack]);
const interviewSource = useMemo(() => interviewQuestions.map((question, index) => ({ question, index })), [interviewQuestions]);
const hasApiKey = apiKey.trim().length > 0;

const persistPacks = (nextPacks: ContentPack[]) => {
  setPacks(nextPacks);
  saveGeneratedPacks(nextPacks);
};

const showToast = (message: string) => setToast(message);
const stopRecognition = () => {
  recognitionRef.current?.stop();
  recognitionRef.current = null;
  setIsListening(false);
  setActiveSpeechTarget(null);
  setVoiceInterim('');
};

const startRecognition = (target: SpeechTarget) => {

```

```

const Recognition = window.SpeechRecognition || window.webkitSpeechRecognition;
if (!Recognition) {
  setVoiceError('Speech recognition is not available in this browser. Use Chrome');
  return;
}
if (isListening) stopRecognition();
const recognition = new Recognition();
recognition.continuous = true;
recognition.interimResults = true;
recognition.lang = 'en-US';
recognition.onstart = () => { setIsListening(true); setActiveSpeechTarget(target); };
recognition.onend = () => { setIsListening(false); setActiveSpeechTarget(null); };
recognition.onerror = (event) => { setVoiceError(event.message || event.error); };
recognition.onresult = (event) => {
  let finalTranscript = '';
  let interimTranscript = '';
  for (let index = event.resultIndex; index < event.results.length; index++) {
    const text = event.results[index][0].transcript;
    if (event.results[index].isFinal) finalTranscript += text;
    else interimTranscript += text;
  }
  if (finalTranscript.trim()) {
    if (target === 'voice') setVoiceTranscript((current) => `${current}${current === '' ? '' : ' '}${finalTranscript}`);
    else setInterviewAnswers((answers) => answers.map((answer, index) => (index < answers.length - 1 ? answer : answer + finalTranscript)));
  }
  setVoiceInterim(interimTranscript);
};
recognitionRef.current = recognition;
recognition.start();
};

const activeSourceContent = inputMode === 'paste' ? pastedContent : inputMode === 'url' ? sourceUrl : '';
const canPrepare = hasApiKey && ((inputMode === 'youtube' && sourceUrl.trim().length > 0) || (inputMode === 'url' && sourceUrl.trim().length > 0));
const buildRequest = () => ({ sourceUrl: inputMode === 'youtube' ? sourceUrl : activeSourceContent, apiKey, audience, style });

const handlePrepareBrief = async () => {
  if (!canPrepare) { showToast('Add source context and API key before generating brief.', 'error'); }
  mirrorPipelineSettings(apiKey, audience, style);
  setIsGenerating(true);
  setPreparation(null);
  setBriefDraft(null);
  try {
    const prepared = await prepareGenerationBrief(buildRequest(), setProgress, setPreparation);
    setBriefDraft(prepared.contentBrief);
    showToast('Brief ready for approval.');
```

```

    setProgress({ stage: 'error', message });
    showToast(message);
  } finally {
    setIsGenerating(false);
  }
};

const handleApproveBrief = async () => {
  if (!preparation || !briefDraft) return;
  setIsGenerating(true);
  try {
    const approved = { ...preparation, contentBrief: { ...briefDraft, approve
    const pack = await generateApprovedContentPack(buildRequest(), approved,
    const nextPacks = [pack, ...packs.filter((item) => item.id !== pack.id)];
    persistPacks(nextPacks);
    setSelectedPackId(pack.id);
    setPreparation(null);
    setBriefDraft(null);
    showToast('War room pack generated.');
```

} catch (error) {
 const message = error instanceof Error ? error.message : 'Generation fail
 setProgress({ stage: 'error', message });
 showToast(message);
 } finally {
 setIsGenerating(false);
 }
 };

 const updateSelectedPack = (pack: ContentPack) => {
 persistPacks(packs.map((item) => (item.id === pack.id ? pack : item)));
 };

 return (
 <div className="min-h-screen bg-[#050505] p-4 text-white lg:p-8">
 <div className="mx-auto max-w-7xl">
 <div className="mb-8 flex flex-wrap items-center justify-between gap-4">
 <div className="grid gap-6 lg:grid-cols-[420px_1fr]">
 <section className="rounded-3xl border border-white/10 bg-white/[0.03]
 <div className="mb-5 grid grid-cols-2 gap-2">{inputModes.map((mode)
 <div className="space-y-4">
 <label className="block text-sm text-white/70">OpenAI API key<Inp
 <label className="block text-sm text-white/70">Audience<Input val
 <label className="block text-sm text-white/70">Theme<Input value=
 <label className="block text-sm text-white/70">Style<select value=
 {inputMode === 'youtube' && <label className="block text-sm text-i
 {inputMode === 'paste' && <label className="block text-sm text-wh
 {inputMode === 'voice' && <div className="space-y-3"><div classNa
 {inputMode === 'interview' && <div className="space-y-3"><div cla

```

        <label className="block text-sm text-white/70">Direction for agen
        <Button className="w-full bg-[#c9a84c] text-black hover:bg-[#d9b9
    </div>
  </section>
  <section className="space-y-6">
    {progress && <div className="rounded-3xl border border-white/10 bg-
    {preparation && briefDraft && <BriefPanel preparation={preparation}
    <div className="rounded-3xl border border-white/10 bg-white/[0.03]
  </section>
</div>
</div>
{selectedPack && <WarRoomWorkspace pack={selectedPack} progressAgents={pr
<Toast message={toast} visible={Boolean(toast)} onClose={() => setToast('
</div>
);
}

```

src/data/types.ts

Plain Text

```

export interface ContentPiece {
  id: string;
  format: string;
  title: string;
  hook: string;
  body: string;
  dm_keyword: string;
  status: string;
}

export interface Asset {
  id: string;
  keyword: string;
  title: string;
  what_is: string;
  exact_content: string;
  how_to: string;
  expected_result: string;
  troubleshooting: string;
}

export interface Prospect {
  id: number;
  name: string;
  company: string;
}

```

```
    industry: string;
    revenue: string;
    linkedin: string;
    website: string;
    icebreaker: string;
    stage: string;
}

export type ContentStyle = 'ai_news' | 'workflow' | 'system';

export type OutputFormat =
  | 'long_post'
  | 'linkedin_post'
  | 'x_thread'
  | 'ig_caption'
  | 'carousel'
  | 'short_script'
  | 'email'
  | 'blog'
  | 'lead_magnet'
  | 'youtube_description'
  | 'conversion_assets';

export type ContentPlatform = 'blog' | 'x' | 'linkedin' | 'instagram' | 'email'
export type PackRating = 'up' | 'neutral' | 'down' | null;
export type PipelineStatus = 'waiting' | 'running' | 'done' | 'error';
export type ClaimStatus = 'verified' | 'weak' | 'unsupported';

export interface AgentLogEntry {
  agent: string;
  summary: string;
  score?: number;
}

export interface TranscriptSegment {
  index: number;
  text: string;
  start?: number;
  duration?: number;
}

export interface VideoTranscriptSource {
  videoId: string;
  url: string;
  title: string;
  source: string;
  language?: string;
  fetchedAt: string;
```

```
    durationSeconds?: number;
    transcript: string;
    segments: TranscriptSegment[];
  }

export interface SourceClaim {
  id: string;
  claim: string;
  quote?: string;
  source: string;
  sourceReference: string;
  status: ClaimStatus;
  category: 'number' | 'quote' | 'story' | 'claim' | 'date' | 'link';
}

export interface ProofSnippet {
  id: string;
  text: string;
  sourceReference: string;
  strength: ClaimStatus;
}

export interface SourceIntelligence {
  id: string;
  createdAt: string;
  primarySourceLinks: string[];
  publishDates: string[];
  exactClaims: SourceClaim[];
  proofSnippets: ProofSnippet[];
  competitorAngles: string[];
  audiencePainLanguage: string[];
  whatEveryoneIsSaying: string[];
  whatLosShouldSayDifferently: string[];
  keyThemes: string[];
  uniqueInsights: string[];
  riskFlags: string[];
  sourceText: string;
  transcriptSource?: VideoTranscriptSource;
}

export interface ContentBrief {
  id: string;
  angle: string;
  targetAudience: string;
  hookPromise: string;
  whyNow: string;
  proofAvailable: string[];
  contentStructure: string[];
}
```

```
    cta: string;
    riskFlags: string[];
    approvedAt?: string;
    editedAt?: string;
  }

export interface PipelineAgentRun {
  key: string;
  name: string;
  status: PipelineStatus;
  produced: string;
  issuesCaught: string[];
  score: number;
  timeSpentMs: number;
  inputSummary?: string;
  outputSummary?: string;
  promptVersion?: string;
}

export interface QualityScores {
  hookStrength: number;
  specificity: number;
  proof: number;
  usefulness: number;
  originality: number;
  voiceMatch: number;
  ctaStrength: number;
  platformFit: number;
  composite: number;
  passed: boolean;
  reasons: string[];
}

export interface WhyThisWorks {
  hookType: string;
  targetDesire: string;
  ctaLogic: string;
  proofUsed: string[];
  audiencePainAddressed: string[];
  platformLogic: string[];
}

export interface CheckedClaim {
  id: string;
  claim: string;
  source: string;
  status: ClaimStatus;
  note: string;
}
```

```
}

export interface WarRoomOutput {
  format: OutputFormat;
  label: string;
  title: string;
  content: string;
  hook?: string;
  cta?: string;
  version: number;
  editedAt?: string;
}

export interface LongPost {
  title: string;
  body_markdown: string;
}

export interface XThread {
  hook: string;
  tweets: string[];
}

export interface IGCaption {
  hook: string;
  body: string;
  cta: string;
}

export interface LinkedInPost {
  hook: string;
  body: string;
  cta: string;
}

export interface EmailAsset {
  subject: string;
  preview_text: string;
  body: string;
  cta: string;
}

export interface YouTubeDescriptionAsset {
  preview: string;
  description: string;
  keywords: string[];
  timestamps: string[];
  links: string[];
}
```

```

}

export interface CarouselSlide {
  title: string;
  bullets: string[];
}

export interface Carousel {
  slides: CarouselSlide[];
}

export interface ShortScript {
  title: string;
  beats: string[];
}

export interface ConversionAsset {
  primary_cta: string;
  soft_cta: string;
  dm_reply_templates: Array<{ scenario: string; reply: string; next_step: string }>;
  paid_offer_push: string;
  pure_value_version: string;
  conversion_optimized_version: string;
  trojan_horse_notes: string[];
  risk_checks: string[];
}

export interface SingleAgentMetadata {
  mode: 'all_agents' | 'single_agent';
  platform?: ContentPlatform;
  engagement_prediction?: {
    initial_score: number;
    reason: string;
    rewritten_hook: string;
    final_score: number;
  };
}

export interface ContentPack {
  id: string;
  tool_name: string;
  source_url: string;
  summary: string;
  audience: string;
  theme: string;
  style: ContentStyle;
  created_at: string;
  posted: boolean;
}

```

```
    long_post: LongPost;
    x_thread: XThread;
    linkedin_post?: LinkedInPost;
    ig_caption: IGCaption;
    email?: EmailAsset;
    youtube_description?: YouTubeDescriptionAsset;
    carousel: Carousel;
    short_script: ShortScript;
    conversion_assets?: ConversionAsset;
    single_agent?: SingleAgentMetadata;
    strategy?: unknown;
    source_brief?: unknown;
    editor_review?: unknown;
    transcript_source?: VideoTranscriptSource;
    agent_log?: AgentLogEntry[];
    quality_score?: number;
    critic_score?: number;
    rating?: PackRating;
    source_intelligence?: SourceIntelligence;
    content_brief?: ContentBrief;
    pipeline_agents?: PipelineAgentRun[];
    quality_scores?: QualityScores;
    why_this_works?: WhyThisWorks;
    checked_claims?: CheckedClaim[];
    prompt_version?: string;
    model?: string;
    editor_notes?: string[];
    war_room_outputs?: WarRoomOutput[];
}
```

```
export interface ApiKeys {
  perplexity: string;
  openai: string;
}
```

```
export interface GenerationSettings {
  apiKeys: ApiKeys;
  audience: string;
  defaultStyle: ContentStyle;
  theme: string;
  voiceTraining?: string;
}
```

```
export interface GenerationRequest {
  sourceUrl: string;
  theme: string;
  style: ContentStyle;
  customPrompt?: string;
```

```

    pastedContent?: string;
    mode?: 'all_agents' | 'single_agent';
    platform?: ContentPlatform;
    offerContext?: string;
}

export interface GenerationProgress {
    stage:
        | 'idle'
        | 'source_intelligence'
        | 'brief'
        | 'research'
        | 'organize'
        | 'optimize'
        | 'write'
        | 'source_check'
        | 'edit'
        | 'tonality_check'
        | 'engagement_check'
        | 'complete'
        | 'error'
        | 'strategizing'
        | 'finding'
        | 'filtering'
        | 'writing'
        | 'quality_gate'
        | 'repurposing'
        | 'conversion'
        | 'editing';
    message: string;
    pack?: ContentPack;
    agents?: PipelineAgentRun[];
    sourceIntelligence?: SourceIntelligence;
    contentBrief?: ContentBrief;
}

export interface SourcePreparation {
    sourceIntelligence: SourceIntelligence;
    contentBrief: ContentBrief;
    agents: PipelineAgentRun[];
}

export const formatLabels: Record<OutputFormat, string> = {
    long_post: 'Long Post',
    linkedin_post: 'LinkedIn',
    x_thread: 'X Thread',
    ig_caption: 'IG Caption',
    carousel: 'Carousel',

```

```

short_script: 'Short Video Script',
email: 'Email',
blog: 'Blog',
lead_magnet: 'Lead Magnet',
youtube_description: 'YouTube Description',
conversion_assets: 'Conversion Assets',
};

export const formatDescriptions: Record<OutputFormat, string> = {
  long_post: 'Flagship long-form operator post with proof and tactical depth',
  linkedin_post: 'Story-driven LinkedIn post with a strong CTA',
  x_thread: 'Seven or more tweets with a proof-led hook',
  ig_caption: 'Hook, body, and CTA for Instagram',
  carousel: 'Slide outline built for saves and shares',
  short_script: 'Short video script with a fast hook and clear beats',
  email: 'Subject, preview text, body, and CTA',
  blog: 'Search-aware blog article with proof and structure',
  lead_magnet: 'Downloadable or DM-worthy asset concept and outline',
  youtube_description: 'SEO description with preview lines, timestamps, and lin
  conversion_assets: 'CTAs, DM replies, offer push, and conversion variants',
};

export const styleLabels: Record<ContentStyle, string> = {
  ai_news: 'AI News',
  workflow: 'Workflow / SOP',
  system: 'Story + System',
};

export const styleDescriptions: Record<ContentStyle, string> = {
  ai_news: 'Update to what changed to use case to how to try it',
  workflow: 'Tactical SOP with numbered steps and screenshots',
  system: 'Story + framework + implementation',
};

```

src/App.tsx

Plain Text

```

import { Routes, Route, Navigate } from 'react-router';
import { Sidebar } from '@components/Sidebar';
import { TopBar } from '@components/TopBar';
import { DashboardView } from '@views/DashboardView';
import { GenerateView } from '@views/GenerateView';
import { MarketRadarView } from '@views/MarketRadarView';
import { IdeaScoringView } from '@views/IdeaScoringView';
import { AgentStudioView } from '@views/AgentStudioView';
import { BrandMemoryView } from '@views/BrandMemoryView';

```

```
import { Settings } from '@views/Settings';

export default function App() {
  return (
    <div className="obsidian-shell min-h-[100dvh]">
      <Sidebar />
      <TopBar />
      <main className="px-4 pb-24 pt-20 lg:pb-10 lg:pl-24 lg:pr-8">
        <div className="mx-auto max-w-[1480px] py-5">
          <Routes>
            <Route path="/" element={<DashboardView />} />
            <Route path="/generate" element={<GenerateView />} />
            <Route path="/market-radar" element={<MarketRadarView />} />
            <Route path="/idea-scoring" element={<IdeaScoringView />} />
            <Route path="/agents" element={<AgentStudioView />} />
            <Route path="/memory" element={<BrandMemoryView />} />
            <Route path="/settings" element={<Settings />} />
            <Route path="*" element={<Navigate to="/" replace />} />
          </Routes>
        </div>
      </main>
    </div>
  );
}
```